



The Tactician

A Seamless, Interactive Tactic Learner and Prover for Coq

Lasse Blaauwbroek^{1,2(✉)}, Josef Urban¹, and Herman Geuvers²

¹ Czech Technical University, Prague, Czech Republic
lasse@blaaubroek.eu, josef.urban@gmail.com

² Radboud University, Nijmegen, The Netherlands
herman@cs.ru.nl

Abstract. We present Tactician, a tactic learner and prover for the Coq Proof Assistant. Tactician helps users make tactical proof decisions while they retain control over the general proof strategy. To this end, Tactician learns from previously written tactic scripts and gives users either suggestions about the next tactic to be executed or altogether takes over the burden of proof synthesis. Tactician’s goal is to provide users with a seamless, interactive, and intuitive experience together with robust and adaptive proof automation.

1 Introduction

The Coq Proof Assistant [3] is an Interactive Theorem Prover in which one proves lemmas using tactic scripts. Individual tactics in these scripts represent actions that transform the proof state of the lemma currently being proved. A wide range of tactics exist, with a wide range of sophistication, from simple inference steps to entire decision procedures and heuristic search procedures.

When proving a lemma, the user’s challenge is to observe the current proof state and select the appropriate tactic and its arguments to be used. Often the user makes this decision based on experience with previous proofs. If the current proof state is similar to a previously encountered situation, then one can expect that an effective tactic in that situation might also be effective now. Hence, the user is continuously matching patterns of proof states in their mind and selects the correct tactic based on these matches.

That is not the only task the user performs, however. When working on a mathematical development, the user generally has two roles: (1) As a *strategist*, the user comes up with appropriate lemmas and sometimes decides on the main structure of complicated proofs. (2) As a *tactician*, the user performs the long and somewhat mindless process of mental pattern matching on proof states, applying corresponding tactics until the lemma is proved. Many of the steps in the tactician’s role will be considered as “obvious” by a mathematician. Our

This work was supported by the European Regional Development Fund under the project AI&Reasoning (reg. no. CZ.02.1.01/0.0/0.0/15_003/0000466) and by the AI4REASON ERC Consolidator grant nr. 649043.

system is meant to replicate the pattern matching process performed in this role, alleviating the user from this burden. Hence, we have aptly named it Tactician.

To perform its job, Tactician can learn from existing proofs, by looking at how tactics modify the proof state. Then, when proving a new lemma, the user can ask the system to recommend previously used tactics based on the current proof state and even to complete the whole proof using a search procedure based on these tactic recommendations.

In our previous publication, we describe technical details on the machine learning techniques employed by Tactician and measure its current automation against Coq’s standard library [1]. This paper instead gives a quick introduction to Tactician from the user perspective. Details on installation and usage of Tactician can be found on the project’s website <http://coq-tactician.github.io>. There, we also explain how Tactician can be used on large projects with complex dependencies.

2 Design Principles

For our system, we start with the principal goal of learning from previous proofs to aid the user with proving new lemmas. In Coq, there are essentially two notions of proof: (1) proof terms expressed in the Gallina language (Coq’s version of CIC [9]); (2) tactic proof scripts written by the user that can then generate a Gallina term. Although it is possible to employ machine learning on both notions, we choose to learn from tactic scripts for two reasons. (1) Tactic scripts are more high-level and forgiving, which is more suitable for machine learning. (2) Working on the tactic level allows the user to introduce domain-specific information to aid the system by writing new tactics. One can teach Tactician about such tactics merely by using them in hand-written proofs a couple of times, after which the system will automatically start to use them.

Apart from the principal goal described above, Tactician’s most important objective is to be usable and remain usable by actual Coq users. To achieve this usability, Tactician needs to be pleasant to all parties involved, which we express in four basic “friendliness” tenets: *user-friendly*, *installation-friendly*, *integration-friendly*, and *maintenance-friendly*. More concretely, it should be usable in any editor, with minimal configuration and no time spent training a ML model. Instead, the system should learn on the fly. Tactician should be tightly integrated with Coq, implemented as a plugin in OCaml without requiring external toolkits. To ensure ease of installation and to prevent it from becoming abandonware, it should be entered into the Coq Package Index [2].

The tight integration with Coq make Tactician function both in Coq’s interactive mode and compilation mode. In the next two sections, we describe how the system is integrated with these modes.

3 Interactive Mode of Operation

We illustrate the interactive mode of operation of Tactician using the schematic in Fig. 1. When the user starts a new Coq development file—say *X.v*—the first

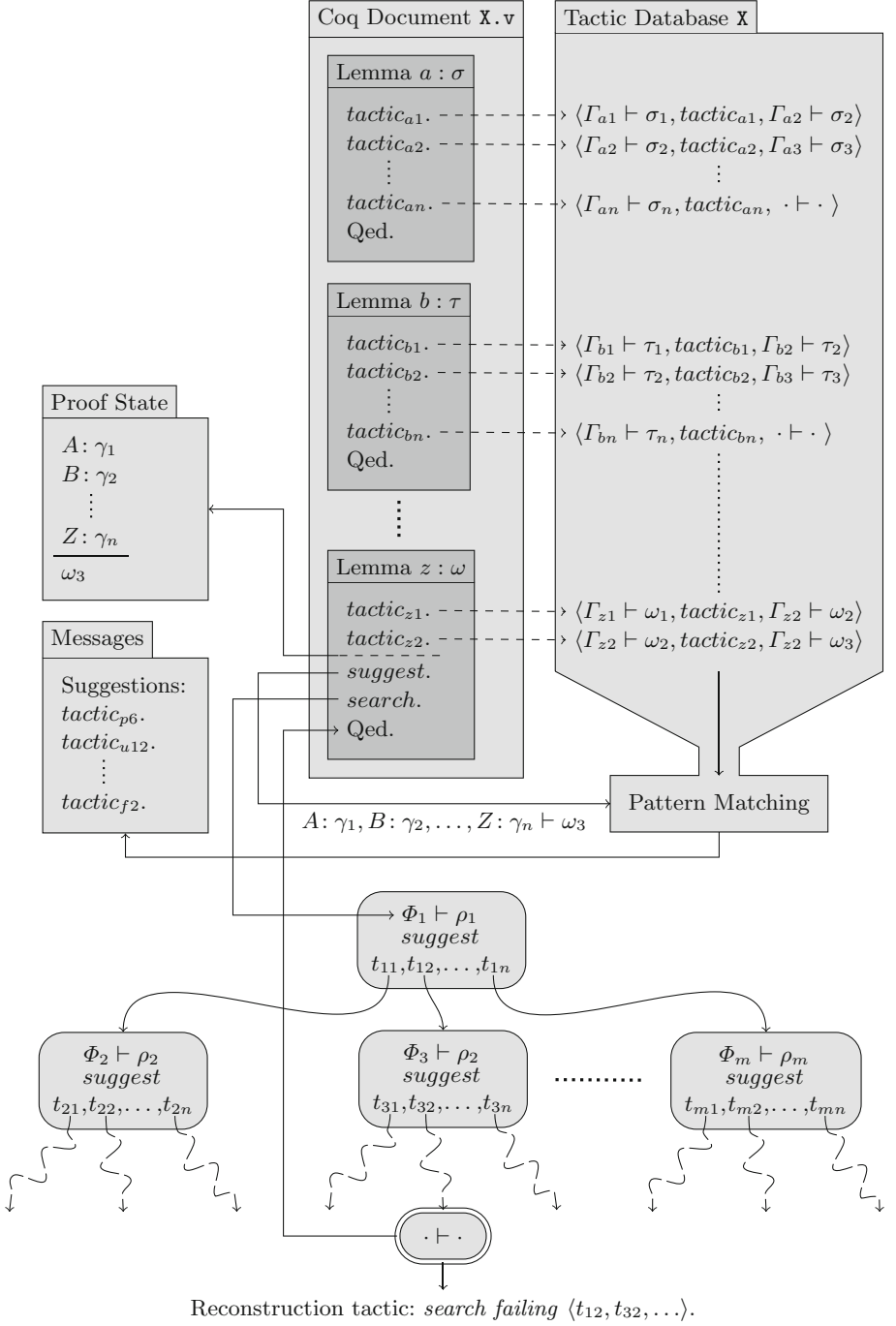


Fig. 1. A schematic overview of Tactician in its interactive mode of operation.

thing Tactician does is create an (in-memory) empty tactic database $\mathbf{X}$ corresponding to this file. The user then starts to prove lemmas as usual. Behind the scenes, every executed tactic, e.g. $tactic_{a1}$, is saved into the database accompanied by the proof states before and after tactic execution, in this case, $\langle \Gamma_{a1} \vdash \sigma_1, tactic_{a1}, \Gamma_{a2} \vdash \sigma_2 \rangle$. The difference between these two states represents the action performed by the tactic, while the state before the tactic represents the context in which it was useful. By recording many such triples for a tactic, we create a dataset representing an approximation of that tactic’s semantic meaning. The database is kept synchronized with the user’s movement within the document throughout the entire interactive session.

After proving a few lemmas by hand, the user can start to reap the fruits of the database. For this, the tactics **suggest** and **search** are available. We illustrate their use in the schematic when “Lemma $z : \omega$ ” is being proven. The user first executes two normal tactics. After that, Coq’s proof state window displays a state for which the user is unsure what tactic to use. Here Tactician’s tactics come in.

suggest. This tactic can be executed to ask Tactician for a list of recommendations. The current proof state $A : \gamma_1, B : \gamma_2, \dots, Z : \gamma_n \vdash \omega_3$ is fed into the pattern matching engine, which will perform a comparison with the states in the tactic database. From this, an ordered list of recommendations is generated and displayed in Coq’s messages window, where the user can select a tactic to execute.

search. Alternatively, the system can be asked to **search** for a complete proof. We start with the current proof state, which we rename to $\Phi_1 \vdash \rho_1$ for clarity. Then a search tree is formed by repeatedly running **suggest** on the proof state and executing the suggested tactics. This tree can be traversed in various ways, finishing only when a complete proof has been found.

If a proof is found, two things happen. (1) The Gallina proof term that is found is immediately submitted to Coq’s proof engine, after which the proof can be closed with **Qed**. (2) Tactician generates a reconstruction tactic **search failing** $\langle t_{12}, t_{32}, \dots \rangle$ which is displayed to the user (see the bottom of the figure). The purpose of this tactic is to provide a modification resilient proof cache that also functions when Tactician is not installed. Normally, the lemma can be reproved using the list of tactics $\langle t_{12}, t_{32}, \dots \rangle$. “Failing” that (due to definitional changes), a new search is initiated to recover the proof. To use the cache, the user should copy it and replace the original **search** invocation with it in the source file.

4 Compilation Mode of Operation

This mode is visualized in Fig. 2. After the file $\mathbf{X.v}$ has been finished, one might want to depend on it in other files. This requires the file to be compiled into a binary $\mathbf{X.vo}$ file. The compilation is performed using the command `coqc X.v`. Tactician is integrated into this process. During compilation, the tactic database is rebuilt in the same way as in interactive mode and then included in the $\mathbf{.vo}$ file.

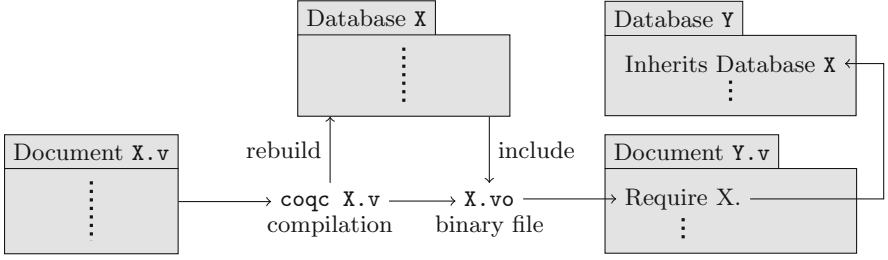


Fig. 2. A schematic overview of Tactician in its compilation mode of operation.

When development $X.v$ is then **Required** by another development file $Y.v$, the tactic database of $X.v$ is automatically inherited.

5 A Concrete Example

We now give a simple example use-case based on lists. Starting with an empty file, Tactician is immediately ready for action. We proceed as usual by giving a standard inductive definition of lists of numbers with their corresponding notation and a function for concatenation.

```

Inductive list :=
| nil : list
| cons : nat -> list -> list.
Notation "[]" := nil.
Notation "x::ls" := (cons x ls).

Fixpoint concat ls1 ls2 :=
match ls1 with
| [] => ls2
| x::ls1' => x::(ls1' ++ ls2)
end where "ls1++ls2" := (concat ls1 ls2).

```

We wish to prove some standard properties of concatenation. The first is a lemma stating that the empty list $[]$ is the right identity of concatenation (the left identity is trivial).

Lemma `concat_nil_r ls : ls ++ [] = ls.`

With Tactician installed, we immediately have access to the new tactics **suggest** and **search**. Neither tactic will produce a result when used now since the system has not had a chance to learn from proofs yet. Therefore, we will have to prove this lemma by hand.

```

Proof. induction ls.
- simpl. reflexivity.
- simpl. f_equal. apply IHls.
Qed.

```

The system has immediately learned from this proof (it was even learning during the proof) and is now ready to help us with a proof of the associativity of concatenation.

Lemma `concat_assoc ls1 ls2 ls3 : (ls1 ++ ls2) ++ ls3 = ls1 ++ (ls2 ++ ls3).`

Now, if we execute `suggest`, it outputs the ordered list `induction ls1, simpl, f_equal, ...`. Indeed, using `induction` as our next tactic is not unreasonable. We can repeatedly ask `suggest` for a recommendation after every tactic we input, which sometimes gives us good tactics and sometimes bad tactics. However, we can also eliminate the middle-man and execute the `search` tactic, which immediately finds a proof.

Proof. `search. Qed.`

To cache the proof that is found for the future, we can copy-paste the reconstruction tactic that Tactician prints into the source file. This example shows how the system can quickly learn from very little data and with minimal effort from the user. Of course, this also scales to much bigger developments.

6 Related Work

Tactician takes its main inspiration from the TacticToe [5] system for HOL4. Our work is similar to TacticToe in principle, but diverges significantly in the implementation details due to the large differences between HOL4 and Coq, both their logical system and practical implementation, see [1].

The most significant distinguishing factor of Tactician to other systems for Coq is its user-friendliness. There are many other interesting ML systems for Coq, such as ML4PG [8], SEPIA [6], GamePad [7], CoqGym [11], and ProverBot9001 [10]. However, all of these systems are either difficult to install, can only be used with one editor, need a long time to train their models or do not have an end-user interface at all. Many such systems are geared towards the AI community rather than towards the Theorem Proving community. CoqHammer [4] is the only other system we know of for Coq that has tight integration with Coq and is directly usable for end-users. For more detailed related work, see [1].

References

1. Blaauwbroek, L., Urban, J., Geuvers, H.: Tactic learning and proving for the Coq proof assistant. In: LPAR23. EPiC Series in Computing, vol. 73, pp. 138–150. EasyChair (2020)
2. Coq Development Team: Coq package index. <https://coq.inria.fr/opam/www>
3. Coq Development Team: The Coq proof assistant, version 8.11.0, October 2019
4. Czajka, L., Kaliszyk, C.: Hammer for Coq: automation for dependent type theory. *J. Aut. Reasoning* **61**(1–4), 423–453 (2018)
5. Gauthier, T., Kaliszyk, C., Urban, J.: TacticToe: Learning to reason with HOL4 tactics. In: LPAR. EPiC Series in Computing, vol. 46, pp. 125–143. EasyChair (2017)
6. Gransden, T., Walkinshaw, N., Raman, R.: SEPIA: search for proofs using inferred automata. In: Felty, A.P., Middeldorp, A. (eds.) CADE 2015. LNCS (LNAI), vol. 9195, pp. 246–255. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-21401-6_16

7. Huang, D., Dhariwal, P., Song, D., Sutskever, I.: Gamepad: a learning environment for theorem proving. In: ICLR (Poster). OpenReview.net (2019)
8. Komendantskaya, E., Heras, J., Grov, G.: Machine learning in proof general: interfacing interfaces. UTP. EPTCS **118**, 15–41 (2012)
9. Paulin-Mohring, C.: Inductive definitions in the system Coq rules and properties. In: Bezem, M., Groote, J.F. (eds.) TLCA 1993. LNCS, vol. 664, pp. 328–345. Springer, Heidelberg (1993). <https://doi.org/10.1007/BFb0037116>
10. Sanchez-Stern, A., Alhessi, Y., Saul, L.K., Lerner, S.: Generating correctness proofs with neural networks. In: arXiv/CoRR. abs/1907.07794 (2019)
11. Yang, K., Deng, J.: Learning to prove theorems via interacting with proof assistants. In: ICML, Proceedings of Machine Learning Research, vol. 97, pp. 6984–6994 (2019)